Sometimes you don’t want the object that’s the receiver of the function. Sometimes you want some other object. That’s what the fat arrow is for.

4.5.2. The fat arrow

Imagine you now want the text on your HTML element to be changed to “Got clicked” after one second, instead of immediately. To do this, you first attach a click handler and then invoke `setTimeout` inside it, passing it a function and a number of milliseconds:

```
someElement.onclick = ->
  setTimeout ->
    @innerHTML 'Got clicked'
  , 1000
```

When this is invoked, @ will refer to the global object.

Try that in your browser; it doesn’t work. That’s because the function being invoked is the one that you gave as an argument to `setTimeout`.

Take a step away from this specific example for a moment and consider it in a more abstract sense. Think of an object with two properties, one being a single method that returns the object itself:

```
noumenon =
  value: 'noumenon',
  itself: -> @
```

Object property that contains string

Object property that contains function

When you invoke the `itself` method, it will return the original object. This means you can get the value by first invoking the `itself` method to get the object back again and then access the property:

```
noumenon.itself().value
# 'noumenon'
```

This example is contrived, but having a method return the object it was called on is a powerful technique (explored in chapter 7). The important thing to consider here is what happens if the function is used somewhere else: